

ORF307

Applications of Linear Optimization

Ioannis Akrotirianakis

Midterm exam #1

- **Date Thursday October 9, 2025**
- **Time: 10:40am – 12.00pm (come 10 minutes earlier!)**
- **Location: 120 Lewis Library**
- **Topics covered: up to Lecture slides 09 and the referenced chapters**
- **Material allowed: single sheet of paper. Double sided. Hand-written or typed**

Today's topics

- Optimal control
- Character recognition
- Portfolio optimization

Optimal control

Optimal control problems

Linear dynamical systems:

$$x_{t+1} = Ax_t + Bu_t$$

$$y_t = Cx_t, \quad t = 1, 2, \dots$$

- $x_t \in R^n \rightarrow$ state vector at time t
- $u_t \in R^m \rightarrow$ input vector at time t
- $y_t \in R^p \rightarrow$ output vector at time t
- $A \in R^{n \times n} \rightarrow$ dynamics matrix
- $B \in R^{n \times m} \rightarrow$ input matrix
- $C \in R^{p \times n} \rightarrow$ output matrix

Simulation

- The sequence $x_1, x_2, x_3 \dots$ is called the **state trajectory**
- The sequence $y_1, y_2, y_3 \dots$ is called the **input trajectory**
- **Goal:** given $x_1, u_1, u_2, \dots$, find $x_2, x_3, \dots$ and $y_2, y_3, \dots$.
- Obtained using **recursion**.
 - For $t = 1, 2, \dots$, compute:

$$x_{t+1} = Ax_t + Bu_t \quad \text{and} \quad y_t = Cx_t$$

Optimal control problem

Linear dynamical systems:

$$x_{t+1} = Ax_t + Bu_t$$

$$y_t = Cx_t, \quad t = 1, 2, \dots$$

The problem

- The **initial state** $x_1 = x^{init}$ is **known**.
- **Goal:** choose $u_1, u_2, \dots, u_{T-1}$ to achieve **some goals**, e.g.,
 - **Goal #1:** Get to **desired final state** $x_T = x^{des}$
 - **Goal #2:** **Minimize the input effort** (make $\|u_t\|$ small for all t)
 - **Goal #3:** **Track desired output** y_t^{des} (e.g., make $\|y_t - y_t^{des}\|$ **small**, $\forall t$)

Least squares optimal control problem (L_2 -norm)

The optimization mode:

$$\begin{aligned} \min \quad & \sum_{t=1}^T \|y_t - y_t^{des}\|_2^2 + \rho \sum_{t=1}^{T-1} \|u_t\|_2^2 \\ \text{s.t.} \quad & x_{t+1} = Ax_t + Bu_t, \quad t = 1, \dots, T-1 \\ & y_t = Cx_t, \quad t = 1, \dots, T \\ & x_1 = x^{init} \end{aligned}$$

Remarks

- The **decision variables** are: $x_2, \dots, x_T$, $y_2, \dots, y_T$ and $u_1, \dots, u_{T-1}$
- Parameter $\rho > 0$ controls the **trade off** between the control energy and the tracking error
- It is a **multi-objective** and **constrained** least squares problem

L_1 -norm optimal control problem

$$\begin{aligned} \min \quad & \sum_{t=1}^T \|y_t - y_t^{des}\|_1 + \rho \sum_{t=1}^{T-1} \|u_t\|_1 \\ \text{s.t.} \quad & x_{t+1} = Ax_t + Bu_t, \quad t = 1, \dots, T-1 \\ & y_t = Cx_t, \quad t = 1, \dots, T \\ & Dx_t \leq d, \quad t = 1, \dots, T \\ & Eu_t \leq e, \quad t = 1, \dots, T \\ & x_1 = x^{init} \end{aligned}$$

Remarks

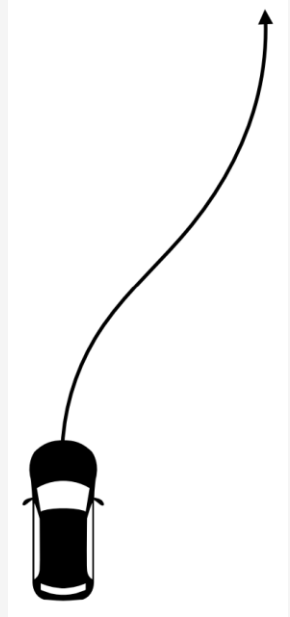
- We use $\|\cdot\|_1$ instead of $\|\cdot\|_2$
- We also have additional linear inequality constraints
 - For states: $Dx_t \leq d$, and for the inputs: $Eu_t \leq e$
- It can be converted to an equivalent LP problem

Vehicle example on a plane

We have a vehicle with **mass** m

We **sample** its **position** and **velocity** at times $t = 0h, h, 2h, 3h, \dots$

- $p_t \in \mathbb{R}^2$: position at time th (t hours after time 0)
- $v_t \in \mathbb{R}^2$: velocity at time th
- $u_t \in \mathbb{R}^2$: **force applied** at time th
- $-\eta v_t$: **friction force** applied at time th



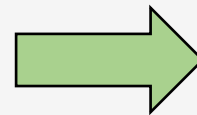
For a small time interval h

Velocity

$$\frac{p_{t+1} - p_t}{h} \approx v_t$$

Newton's 2nd Law
 $m \times a \approx F_{net}$

$$m \frac{v_{t+1} - v_t}{h} \approx -\eta v_t + u_t$$



$$p_{t+1} = p_t + hv_t$$

$$v_{t+1} = (1 - h\eta/m)v_t + (h/m)u_t$$

Vehicle example on a plane

State

$$x_t = (p_t, v_t) \in R^4$$

Dynamics

$$x_{t+1} = Ax_t + Bu_t$$

$$y_t = Cx_t$$

Laws of Physics

$$p_{t+1} = p_t + hv_t$$

$$v_{t+1} = (1 - h\eta/m)v_t + (h/m)u_t$$

$$\text{Output = Position} \rightarrow y_t = p_t$$

$$A = \begin{bmatrix} 1 & 0 & h & 0 \\ 0 & 1 & 0 & h \\ 0 & 0 & 1 - h\eta/m & 0 \\ 0 & 0 & 0 & 1 - h\eta/m \end{bmatrix},$$

$$B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ h/m & 0 \\ 0 & h/m \end{bmatrix},$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Vehicle example with output tracking

Least squares results (L_2 -norm)

Parameters

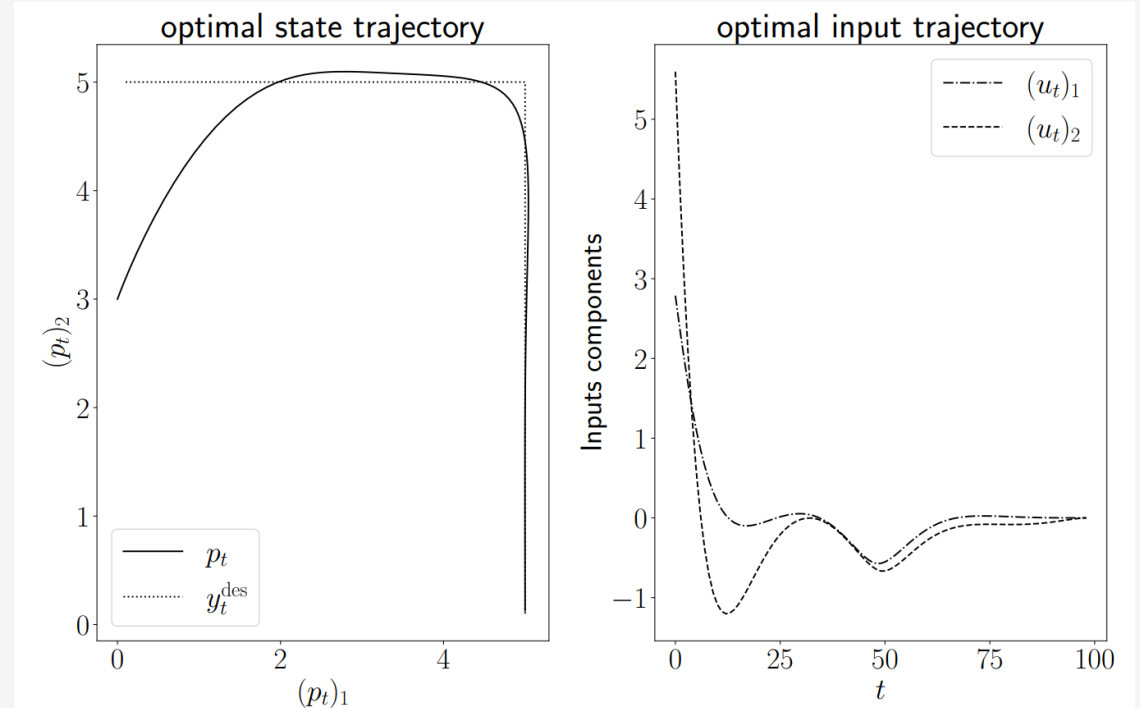
$$T = 100, \quad h = 0.1, \quad \eta = 0.1, \quad m = 1$$

Observations:

Smoothness: The ℓ_2 -norm encourages gradual, energy-efficient control strategies.

Precision: The vehicle closely tracks the desired trajectory.

Trade-off: While smooth, this approach may be less robust to outliers or sudden changes in the desired path.



Discourages abrupt changes and large magnitudes in the input.

Vehicle example with output tracking

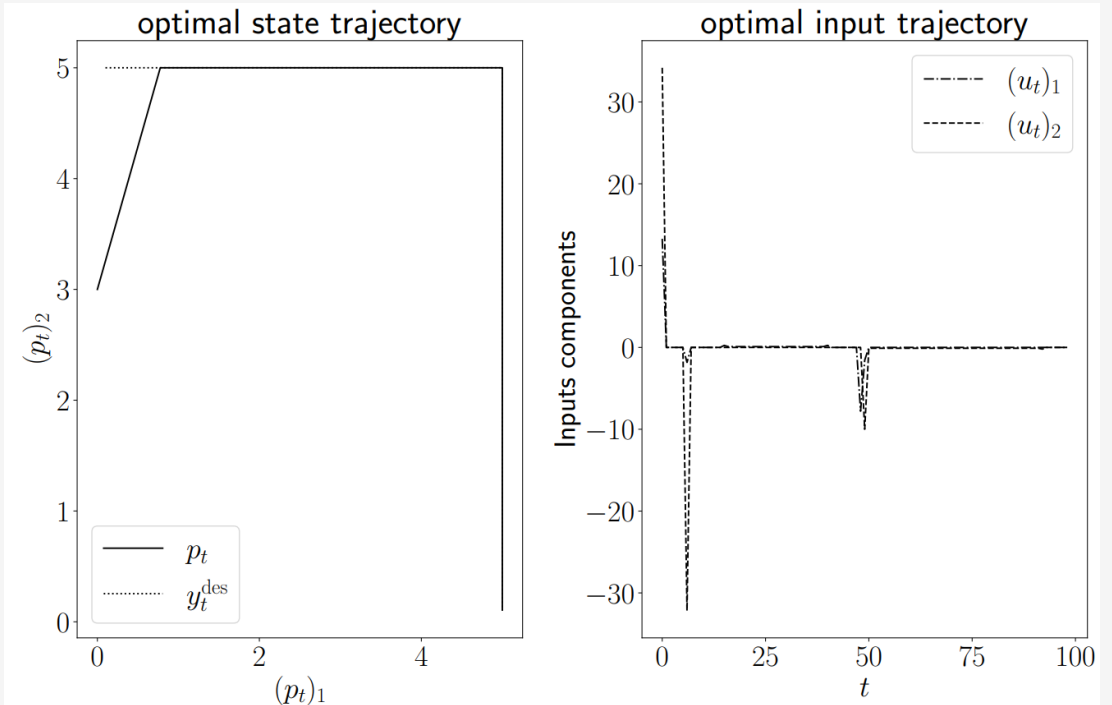
ℓ_1 -norm results

Parameters

$$T = 100, \quad h = 0.1, \quad \eta = 0.1, \quad m = 1$$

Observations:

- **Efficiency:** The ℓ_1 -norm solution (the controller) uses fewer actions to guide the vehicle.
- **Robustness:** It's less sensitive to noise or erratic desired trajectories.
- **Trade-off:** While it may not achieve perfect smoothness, it will consume less fuel than the ℓ_2 -norm trajectory



They jump at key moments and remain zero or constant elsewhere.

Minimal, targeted interventions rather than continuously adjusting the input

Vehicle example with output tracking

ℓ_1 -norm results with constraints

Linear Optimization can introduce interesting constraints

$$\min \sum_{t=1}^T \|y_t - y_t^{des}\|_1 + \rho \sum_{t=1}^{T-1} \|u_t\|_1$$

$$\text{s.t. } x_{t+1} = Ax_t + Bu_t, \quad t = 1, \dots, T-1$$

$$y_t = Cx_t, \quad t = 1, \dots, T$$

max-input



$$\|u_t\|_\infty \leq u^{max}, \quad t = 1, \dots, T-1$$

max-input
variation



$$\|u_t - u_{t-1}\|_1 \leq s^{max}, \quad t = 1, \dots, T-1$$

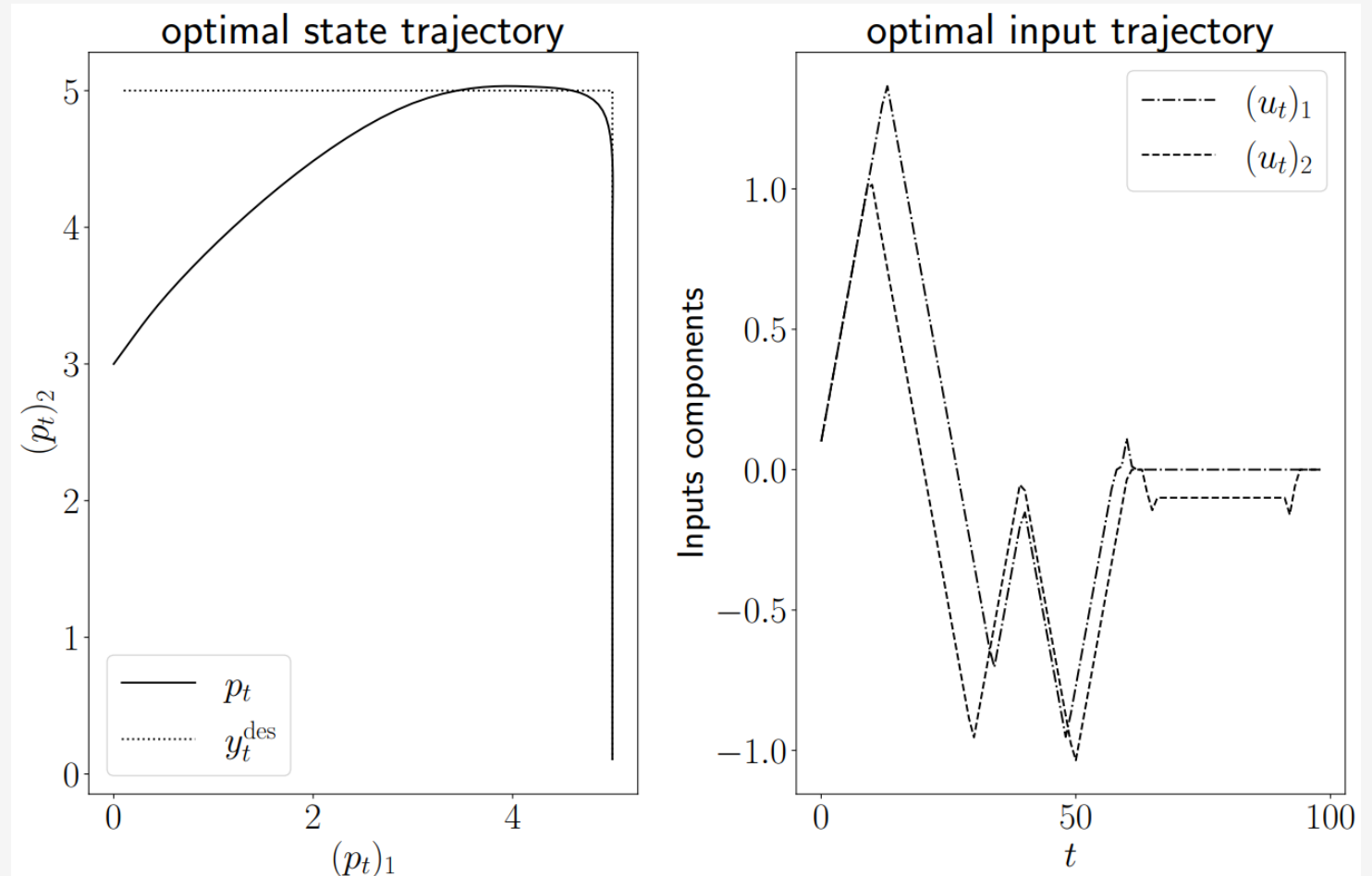
$$x_1 = x^{init}$$

Vehicle example with output tracking

ℓ_1 -norm results with constraints

Parameters

$$u^{max} = 10, s^{max} = 0.1$$



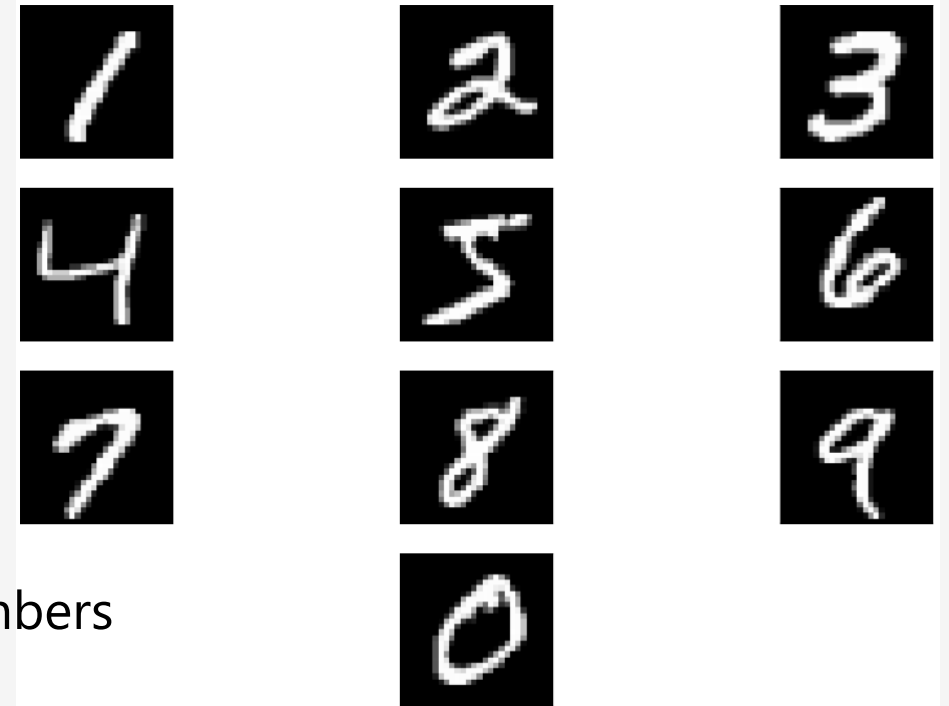
Character recognition

Character recognition

MNIST dataset of handwritten numerals

- Each character is 28×28 pixels
- 60,000 example images
- 10,000 further testing images
- Each sample comes with a label: 0 – 9

Goal: Use linear classification to identify handwritten numbers



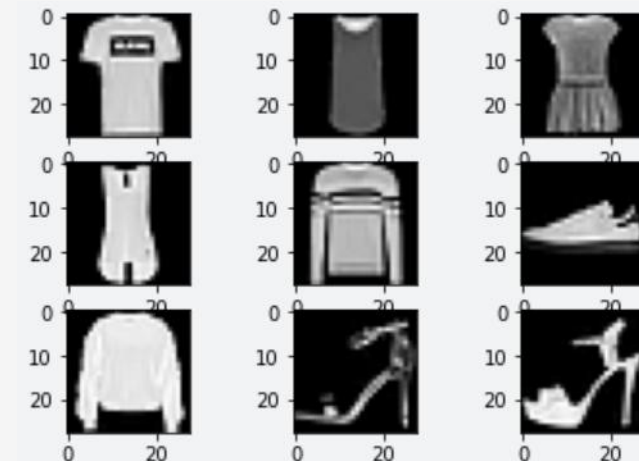
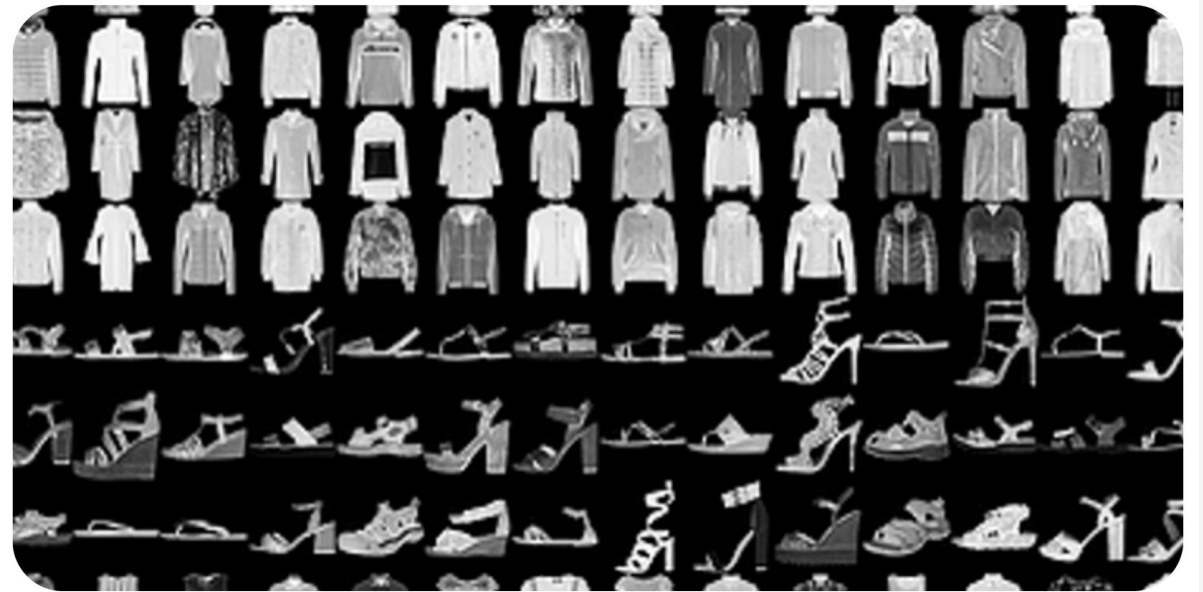
MNIST stands for: **M**odified **N**ational Institute of **S**tandards and **T**echnology

Widely used for training and testing in new Machine Learning algorithms and models

The numbers are **digitized handwritten numbers** written by **school kids** and **US Census employees**

Character recognition - extensions

- There is also the **Fashion-MNIST** dataset
- More challenging than MNIST
- It contains 70,000 grayscale images in total
 - 60,000 for training,
 - 10,000 for testing
- Each image is small, only 28×28 pixels.
- There are 10 unique classes
 - T-shirt/top, Trouser, Pullover, Dress, Coat,
 - Sandal, Shirt, Sneaker, Bag, Ankle boot
- **Who said nerds are not fashionable?!**



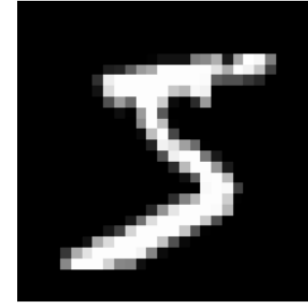
Representing the images

Monochrome images

- **Images** are represented as an $m \times n$ **matrix** X
- Each matrix **element** X_{ij} is a **pixel's intensity** (0=black, 255=white)
- We can also represent a matrix $X \in R^{m \times n}$ by an mn -dimensional **vector** $x \in R^{mn}$

$$X_{ij} = x_k, \text{ where } k = m(j - 1) + i$$

$X =$



(in MNIST, $m = n = 28$)

$x =$



Linear classification

Support Vector Machine (linear separation)

- Given a set of points $\{v_1, v_2, \dots, v_N\}$ and their binary labels $s_i \in \{1, -1\}$
- Goal:** Find a hyperplane that strictly separates the two classes

$$a^T u_i + b > 0 \text{ if } s_i = 1$$

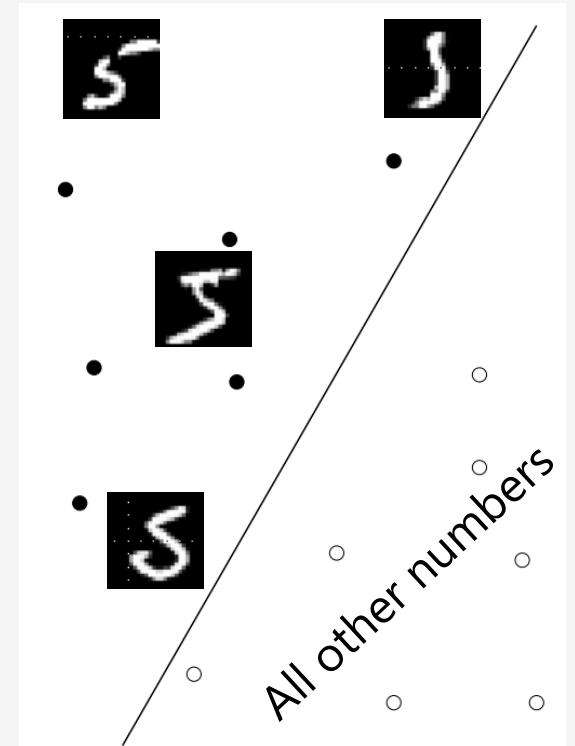
$$a^T u_i + b < 0 \text{ if } s_i = -1$$



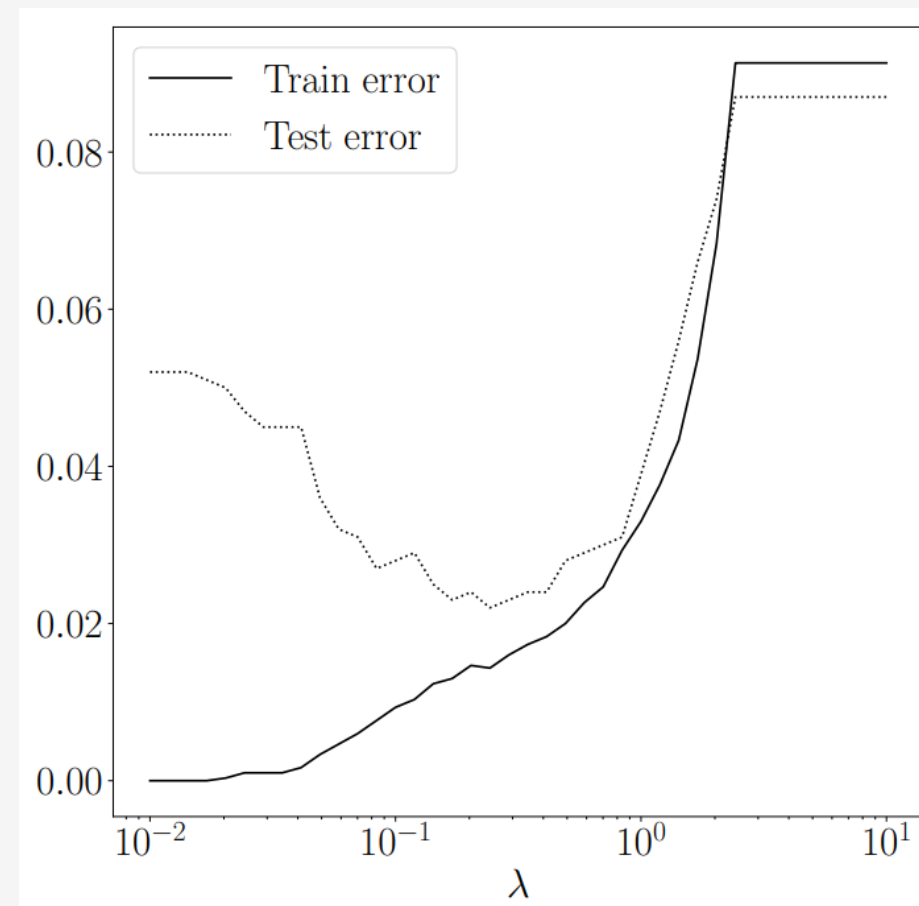
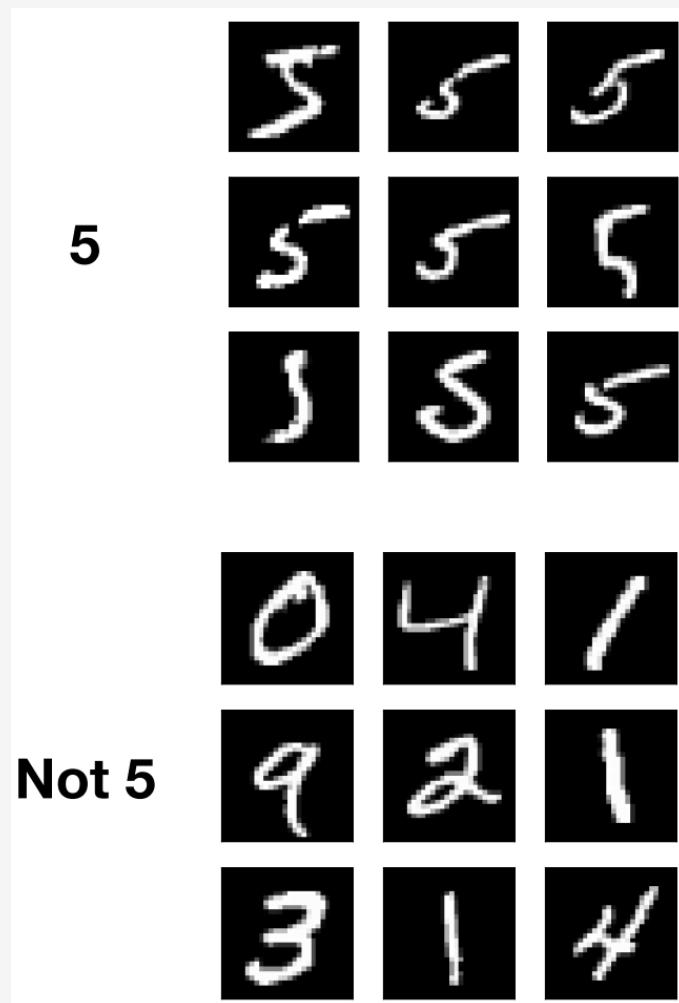
$$s_i(a^T u_i + b) > 1$$

- Minimize the sum of violations + regularization

$$\min \sum_{i=1}^N \max\{0, 1 - s_i(a^T v_i + b_i)\} + \lambda \|a\|_1$$



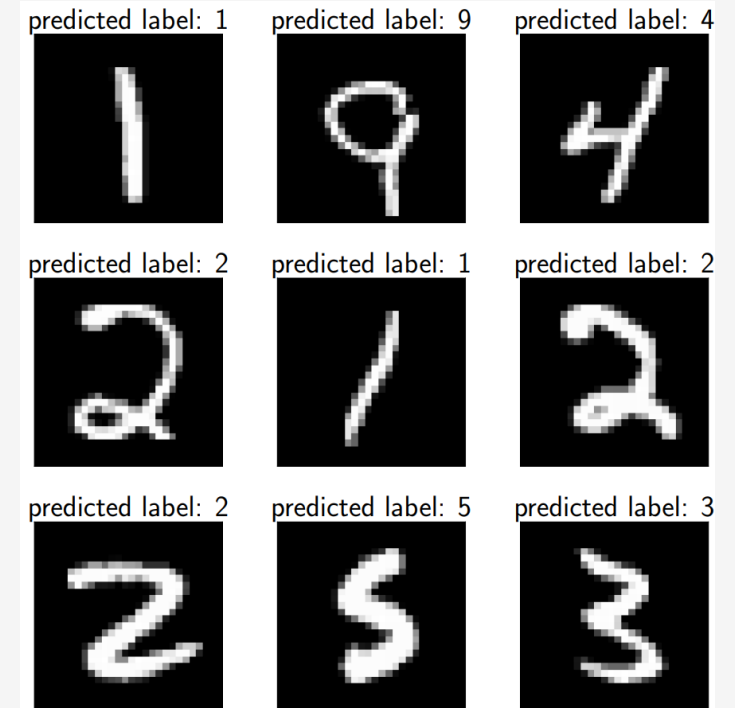
Learn to classify 5



Multiclass classification

One-vs-All

1. **Train one classifier per label** $k \in \{0, 1, \dots, 9\}$ (e.g., k versus anything else),
 - **You will obtain 10 models:** $(a_k, b_k), k = 0, \dots, 9$
2. Feed an **unclassified & unknown** image $v^{(i)}$
3. **Predict all results:** $r^{(k)} = a_k^T v^{(i)} + b_k, k = 0, \dots, 9$
4. **Take the index corresponding to the maximum** $r^{(k)}$
 - $\ell = \arg \max \{r^{(k)} : k = 0, \dots, 9\}$
5. The label of **image** $v^{(i)}$ is ℓ
 - Image $v^{(i)}$ corresponds to the number ℓ (e.g., 2)



Portfolio optimization

Portfolio allocation weights

We have a **capital** of V dollars and want to **invest** it in n different assets over periods $t = 1, \dots, T$

$w \in \mathbb{R}^n$: the vector of the fractions allocated in each of the n assets

Individual investments:

Investment in asset i (in dollars): Vw_i , $\forall i$

Normalization: $\mathbf{1}^T w = 1$

If $w_j < 0$ then you have a short position (you borrowed money, and must return it at time T)

Example: $w = (-0.2, 0.0, 1.2)^T$

- Short position on asset 1
- No investment on asset 2
- Long position on asset 2

Return over a time period

Asset returns:

$\tilde{r}_t \in R^n$: the (fractional) return vector holding the return of each asset over period t

Example:

$\tilde{r}_t = (0.01, -0.023, 0.02)^T$ as fractional returns

$\tilde{r}_t = (1\%, -2.3\%, 2\%)^T$ as percentage returns

Portfolio return:

$$r_t = \tilde{r}_t^T w$$

The fractional return of the entire portfolio over period t

Total portfolio value after one period:

$$V_{t+1} = V_t + V_t r_t = V_t(1 + r_t)$$

Portfolio optimization

The Question: how should we choose the portfolio weight vector w ?

Two main goals:

1. High (average) returns
2. Low risk

What we know (data):

- The realized (or historical) asset returns
- Estimate historical risk of the assets

Optimization: we choose the optimal weights (w) based on historical data

Our hope: the weights will be valid for the near future

Linear optimization for portfolio optimization

Average asset returns

$$\text{avg}(r) = \frac{1}{T} \mathbf{1}^T (Rw) = \frac{1}{T} (R^T \mathbf{1})^T w = \mu^T w \quad \mu \in R^n \text{ is the vector of average returns}$$

Benefits:

- **Less sensitive** to large deviations (outliers)
 - **Large outliers lead to conservative portfolios**
- No longer need for $\text{std}(r)$ (divided by T)
 - More **interpretable** to investors than $\text{std}(r)$
- Can be represented as an **LP**
- Produces **sparse fluctuations**: $|r_i - \text{avg}(r)|$

ℓ_1 -norm risk approximation

$$\frac{1}{T} \|r - \text{avg}(r) \mathbf{1}\|_1$$

Risk-return objective

$$-\mu^T w + \lambda \frac{1}{T} \|Rw - (\mu^T w) \mathbf{1}\|_1$$

Portfolio optimization

Minimize risk-return tradeoff

Choose the weight vector $w \in R^n$ by solving

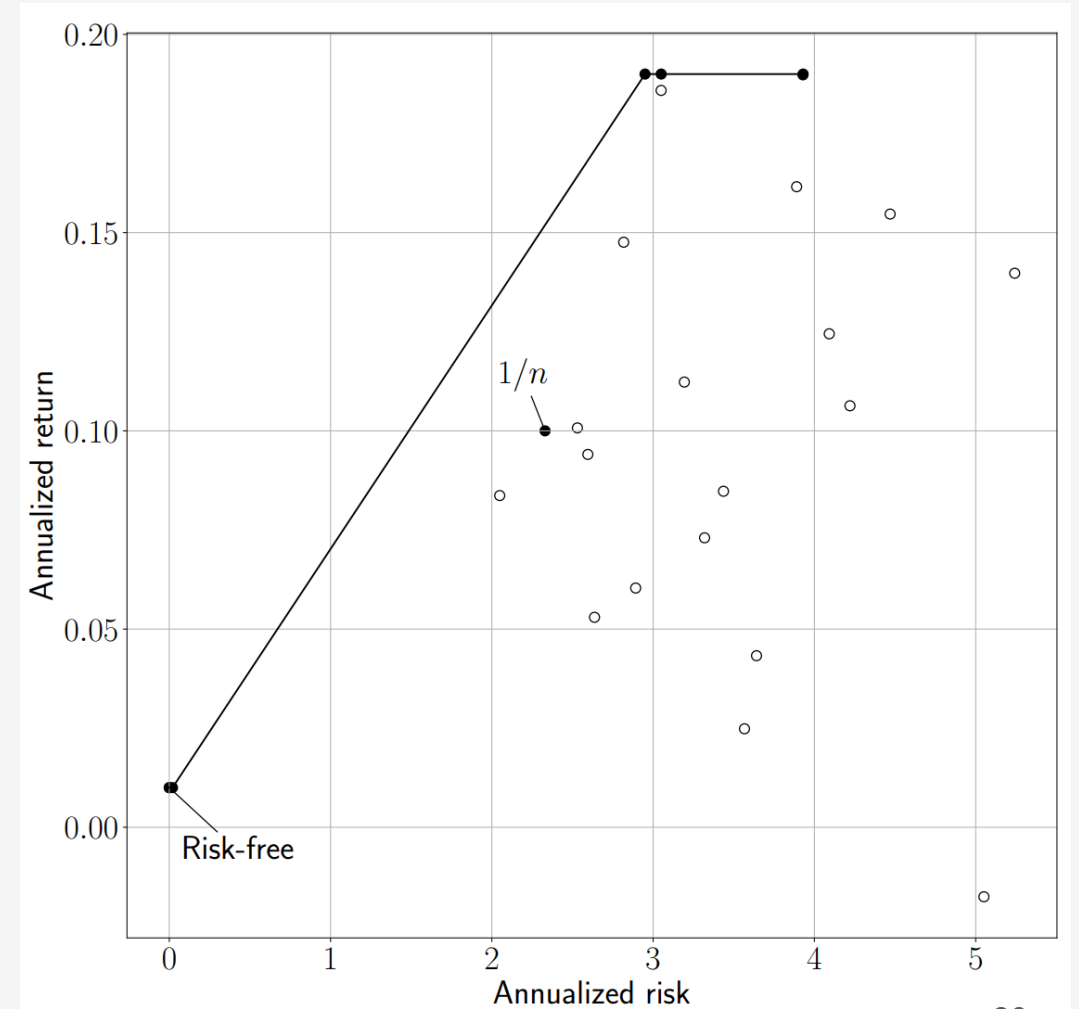
$$\begin{aligned} \min \quad & -\mu^T w + \lambda \frac{1}{T} \|Rw - (\mu^T w)\mathbf{1}\|_1 \\ \text{s.t.} \quad & \mathbf{1}^T w = 1 \\ & w \geq 0 \end{aligned}$$

Remarks

- Can have **inequality constraints** (e.g., long only)
- **Tune** the tradeoff parameter λ to get desired Pareto-optimal solution
- Gives the **best allocation** w^* given past returns

Example

- Optimal portfolios on a straight line
- The line starts at the risk-free portfolio ($\lambda = 0$)
- The $1/n$ portfolio performs much better than single portfolios



The big assumption (and deception)

By using historical returns, we assume that the future returns will look like past ones.

Common practice:

- Compute w by using historical data of the past 2-5 years.
- Then use w for the next 6 months
- After 6 months compute a new w using more recent historical data

But in general, **the assumption is not true in many cases.**

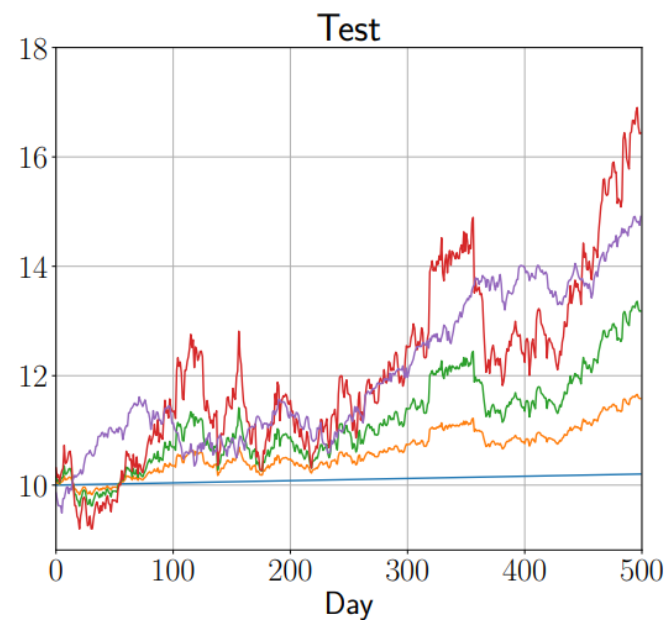
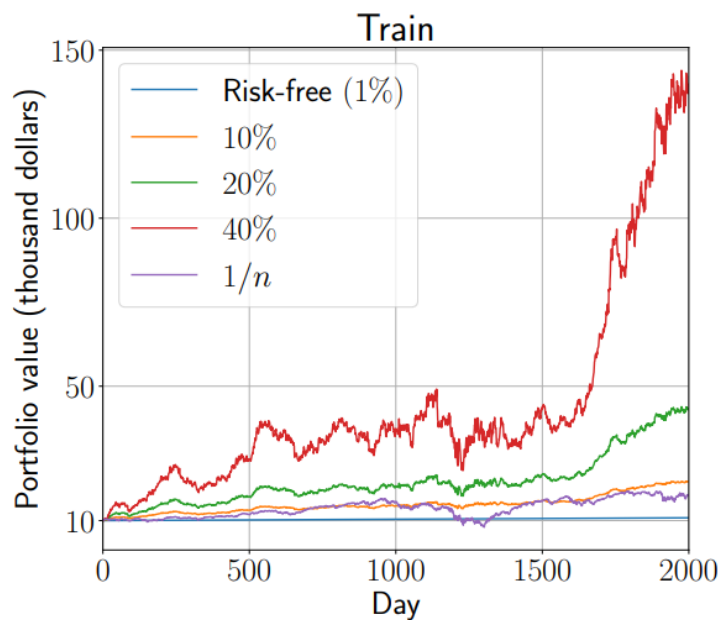
- Especially when **global events** (e.g., COVID) happen that are **not captured** in historical data

The **SEC** requires the disclaimer to be included in investment products:

“Past performance is not guarantee for future results!”

Total portfolio value

	Return		Risk		Leverage
	Train	Test	Train	Test	
Risk-free (1%)	0.01	0.01	0.00	0.00	1.00
10%	0.10	0.08	0.09	0.07	1.96
20%	0.20	0.15	0.18	0.15	3.03
40%	0.40	0.30	0.37	0.31	5.48
1/n	0.10	0.21	0.23	0.13	1.00



Build your own quantitative hedge fund

Rolling portfolio optimization

For each time-period t , compute weights w_t by looking N periods back

$$r_{t-1}, r_{t-2}, \dots, r_{t-N}$$

Other variations

- Update w every K periods (e.g., monthly, quarterly, ...)
- Add a **secondary objective** $\lambda ||w_t - w_{t-1}||^2$ to discourage turnover and reduce transaction costs
- Add **predictive logic** to detect when the future is likely to not look like the past
- **Add “signals”** that predict future returns of assets (e.g., Twitter sentiment analysis)

References

- Study the Github companion notebooks

Next lecture

- The Simplex method